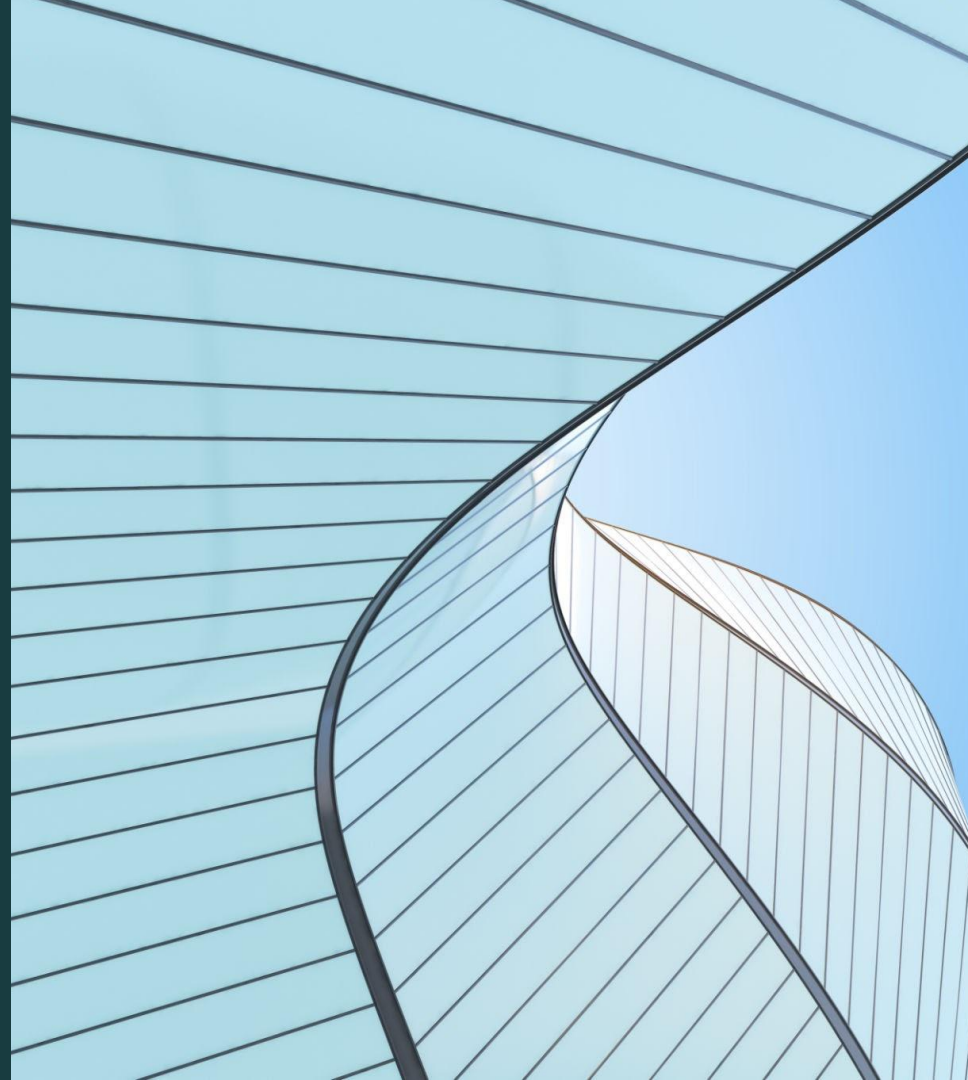


Skin Cancer Detection Milestone 1

By : Samuel Borges, Elif Dila Bayrak,
Neva Ildem, Sophia Zubieta



Progress Matrix Milestone 1

Task	Completion %	Elif	Neva	Samuel	Sophia	To do
Compare and Select Technical Tools	100%	25%	25%	25%	25%	none
“Hello World” Demos	100%	25%	25%	25%	25%	none
Resolve Technical Challenges	100%	25%	25%	25%	25%	none
Compare and Select Collaboration Tools	100%	25%	25%	25%	25%	none
Requirements Document	100%	25 %	25%	25%	25%	none
Design Document	100%	25%	25%	25%	25%	none
Test Plan	100%	25%	25%	25%	25%	none

Technical Tools Selection

Image Preprocessing : Python & OpenCV

Machine Learning Frameworks : TensorFlow/Keras

Evaluation Metrics: Scikit-Learn

Application Interface: Flutter, Android Studio, Dart, SQLite

Collaboration & Version Control : GitHub, GitHub Pages, Google Colab

Functional Requirements

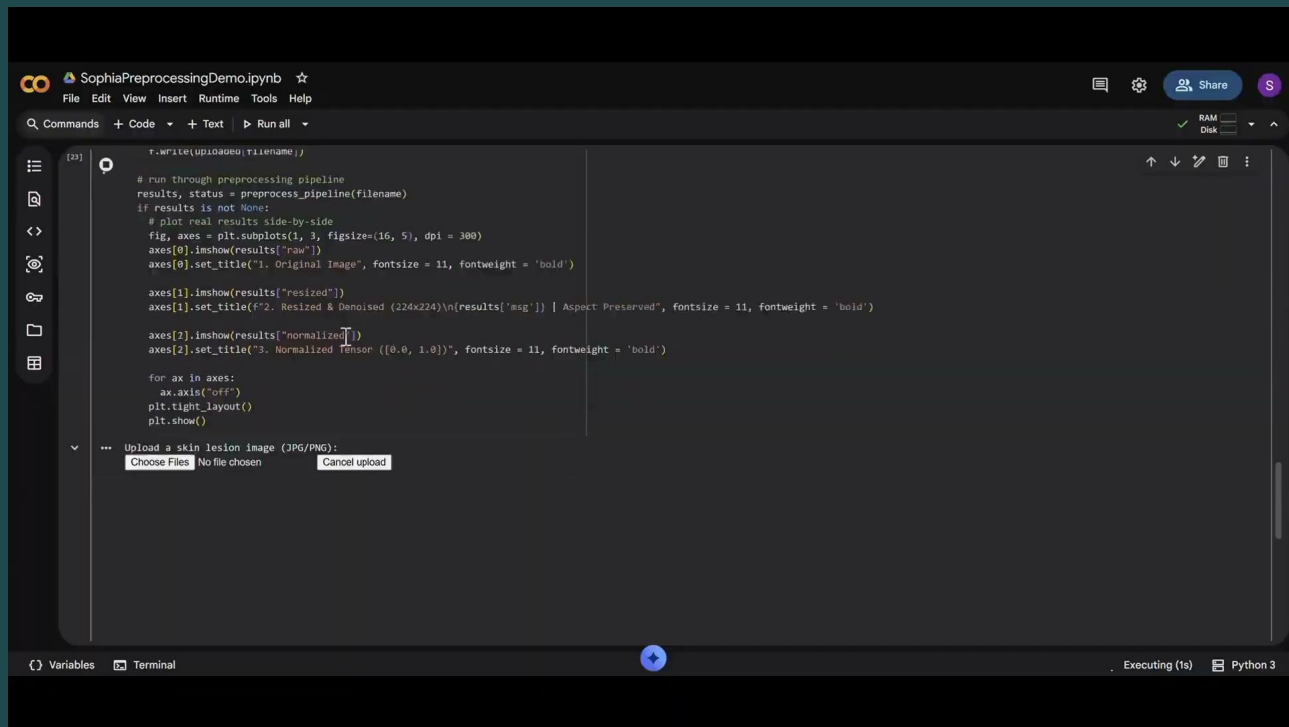
- Image Input
 - Users may upload an image via the device's camera or gallery.
- Image Preprocessing
 - Process the image to match the format and requirements of training ISIC datasets.
- Machine Learning Model Training
 - Train CNN models using skin-lesion datasets.
- Machine Learning Model Evaluation
 - Evaluate the trained models using metrics such as accuracy, precision, and recall.
- Skin Lesion Prediction
 - Provide predictions of skin lesions (Benign or Malignant).
- Prediction Confidence Visualization
 - Provide a percentage of how certain the model is regarding prediction accuracy.
- Training Data Comparison
 - Compares images with thousands of images from the trained datasets.
- Safety Information and Disclaimers
 - Informs users that results are not a medical diagnosis to ensure safety.
- History and Lesion Tracking
 - Tracking the progress of the lesion through multiple image sessions.
- Data Management
 - Store and organize the user's account information and images, and implement security measures.
- Application Interface
 - Provide a simple application interface for easy navigation.

Non-functional Requirements

- Performance
 - Preprocess the image and provide the prediction in a short amount of time
- Scalability
 - Supports additional users, images and different features
- Accuracy
 - Model accurately identifies unseen images
- Reliability
 - Consistently produce results without crashing or handling errors
- Usability
 - Simple interface and intuitive for users with no technical experience
- Security
 - Protect data from unauthorized access
- Maintainability
 - Reliable code and structures to ensure easy modification of code if needed

Image Preprocessing Pipeline Demo

- Simple hello world demo
 - user able to upload image of their choice to get preprocessed
 - resizes image without losing aspect ratio
 - denoises image
 - without losing detailing
 - Normalizes image for deep learning model use
 - Shows output for each step of preprocessing



Machine Learning & Pipeline Experiments

- Pipeline:
 - CIFAR-10 → Train/Validation/Test Split → Model Training → Model Evaluation → Compare Results
- Preliminary ML Pipeline Demo:
 - Implemented and tested a complete model-training pipeline
 - Evaluated basic CNN, Resnet50, EfficientNetB0, and MobileNetV2
 - Used CIFAR-10 to verify training, validation, model saving, and evaluation workflow

```
# Test the trained CNN using the test images
test_loss, test_accuracy = model.evaluate(x_test, y_test)

# Show the final results
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

313/313 ————— 1s 3ms/step - accuracy: 0.6608 - loss: 1.0306
Test Loss: 1.030598759651184
Test Accuracy: 0.6607999801635742
```

```
# Testing the trained ResNet50 model using the untouched CIFAR-10 test images.
resnet50_test_loss, resnet50_test_accuracy = resnet50_model.evaluate(
    x_test,
    y_test
)
```

```
# Displaying the final ResNet50 test results.
print("ResNet50 Test Loss:", resnet50_test_loss)
print("ResNet50 Test Accuracy:", resnet50_test_accuracy)
```

```
313/313 ————— 5s 15ms/step - accuracy: 0.3750 - loss: 1.8309
ResNet50 Test Loss: 1.8308974504470825
ResNet50 Test Accuracy: 0.375
```

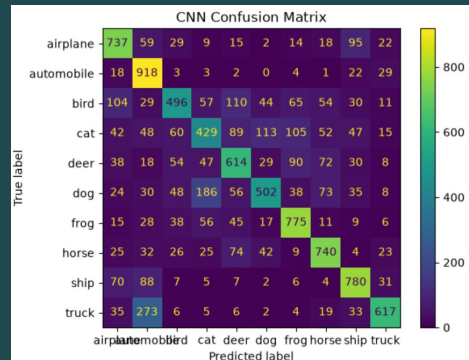
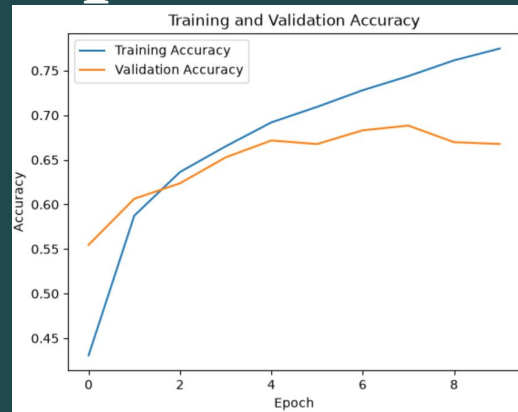
```
# Test the trained EfficientNetB0 model using the test images.
efficientnet_test_loss, efficientnet_test_accuracy = best_efficientnet_model.evaluate(
    x_test,
    y_test
)
```

```
# Show the final results.
print("EfficientNetB0 Test Loss:", efficientnet_test_loss)
print("EfficientNetB0 Test Accuracy:", efficientnet_test_accuracy)
```

```
313/313 ————— 13s 23ms/step - accuracy: 0.1360 - loss: 2.3345
EfficientNetB0 Test Loss: 2.3345117568969727
EfficientNetB0 Test Accuracy: 0.13600000739097595
```

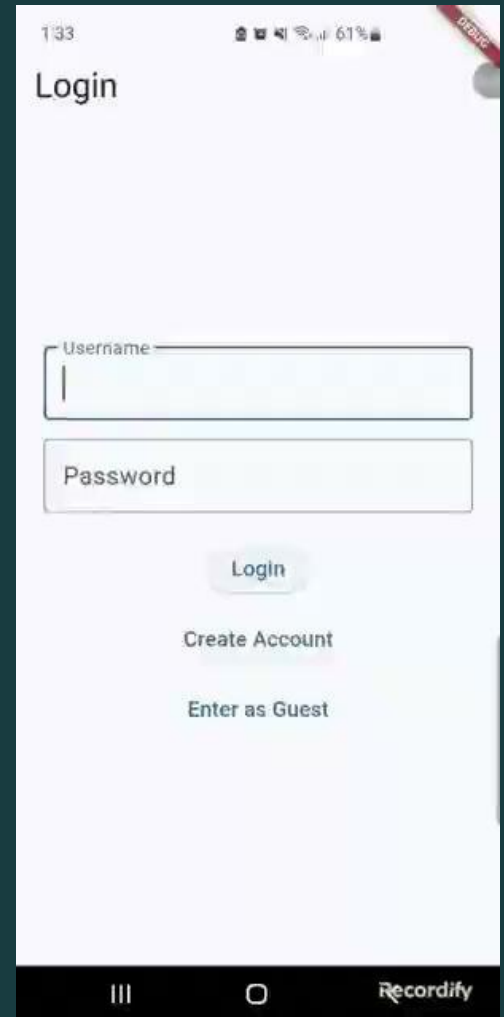
MobileNetV2 Test Loss: 123.85963439941406

MobileNetV2 Test Accuracy: 0.09669999778270721



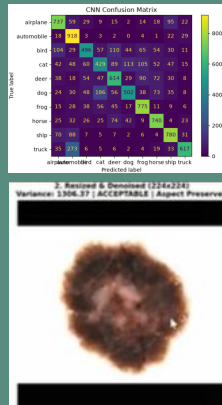
Application Interface Demo

- Flutter mobile application
- Dart used to develop screens, buttons and application logic
- SQLite used to store and retrieve user account information
- Android Studio used to provide Android development tools, deployment and testing on mobile devices



Technical Challenges

- Machine Learning Model Selection
 - Compared basic CNN and transfer learning architectures.
 - Final model selection requires experiments on skin lesion dataset.
- Model Performance Evaluation
 - Accuracy alone is not sufficient.
 - Precision, recall/sensitivity, F1 score, confusion matrices, and false negatives will be evaluated.
- Image Quality & Preprocessing
 - Lesion images may vary in size, quality, noise, and appearance, requiring consistent preprocessing before inference.
- Mobile App & ML Integration
 - The Android application must securely send images to the server/API and receive classification and confidence results.



Milestone 2 Task Matrix

Task	Elif	Neva	Samuel	Sophia
Dataset & Preprocessing Pipeline Execution	-	Prepare dataset for model training.	-	Execute preprocessing pipeline.
CNN Model Training	Train and compare CNN models.	Train and compare CNN models.	-	-
Mobile App Frontend UI	-	-	Develop Android UI.	-
REST API Backend & Preprocessing Endpoint	-	-	-	Develop REST API & preprocessing endpoint.
Model Evaluation & Confusion Matrix	Evaluate models & metrics.	Evaluate models & metrics.	-	-
App to Server Image Upload	Support server connection and testing.	-	Implement image upload from app.	-

Thank you!